William Jackson

Senior Software Engineer - React Python (Django/FastAPI) AWS

Wellington, Nevada, United States | +1 (786) 949-7561| williamjackson.pro@outlook.com | linkedin.com/in/william-jackson-75b9343b6

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years building **Python** and **React** solutions across **network operations**, **data products**, and **reliability-focused SaaS**, where uptime, latency, and auditability directly affect customer trust and revenue. Strong hands-on experience delivering **Python 3.8–3.11**, **FastAPI**, **Django**, **React 16–18**, and **TypeScript** systems behind stable **REST** and **GraphQL** contracts, with a production mindset spanning architecture, implementation, incident response, and postmortem-driven hardening. Known for shipping replay-safe workflows, modernizing legacy services without breaking downstream teams, and improving visibility through **OpenTelemetry**, **Datadog**, **Prometheus**, and **Grafana** across AWS-first environments.

SKILLS

- **Frontend** — **React 16–18**, **TypeScript 4–5**, JavaScript, HTML5, CSS3, Tailwind CSS, Redux Toolkit, React Query, component-driven UI, SPA architecture, state management, performance tuning, form handling, accessibility
- **Backend & APIs** — **Python 3.8–3.11**, **FastAPI**, **Django**, SQL, REST APIs, GraphQL, microservices, event-driven architecture, background jobs, WebSockets, rate limiting, idempotency, caching strategies, Bash
- **Data, Messaging & Search** — PostgreSQL, MySQL, SQL Server, Redis, MongoDB, DynamoDB, AWS SQS/SNS, Kafka, RabbitMQ, Elasticsearch, OpenSearch, Redshift, BigQuery
- **Cloud & DevOps** — AWS (EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, ElastiCache, EventBridge, Step Functions, IAM), Azure (AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights), GCP (GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery), Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, CI/CD (GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI)
- **Observability, Quality & Security** — OpenTelemetry, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch, structured logging, Jest, React Testing Library, Cypress, Playwright, API contract tests, OAuth2, OpenID Connect, JWT, Okta, Auth0, AWS Cognito, RBAC/ABAC, audit logging

WORK HISTORY

Senior Software Engineer
February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Led delivery of a **Python 3.11** and **React 18** platform where **FastAPI** services and component-driven operational dashboards shipped behind stable contracts, bounded payloads, and consistent error handling that reduced cross-team integration friction.
- Fixed a critical double-billing issue caused by duplicate events by enforcing idempotency keys in **SQS FIFO**, adding conditional writes in **DynamoDB**, and exposing replay diagnostics through a **React** incident console used by support and operations.
- Resolved a hard-to-reproduce latency spike by replacing N+1 database access with batched queries on **PostgreSQL 13**, then surfaced query health and degraded workflow states in a **React** admin view for faster operational triage.
- Upgraded legacy workers from **Celery 4.4** to **Celery 5.3**, introducing task routing, safer retries, and visibility-timeout controls, while building **React** monitoring screens that showed queue lag, retry patterns, and poison-message trends.
- Implemented a new deterministic replay capability using **Step Functions** and durable checkpoints, then delivered a **React 18** workflow review interface so operators could inspect, approve, and safely re-run failed automations without duplicate outcomes.
- Reduced noisy paging by migrating ad-hoc logs into **OpenTelemetry 1.x** traces and SLO-based alerts in **Datadog**, while wiring trace-linked incident drilldowns into internal **React** tooling used during on-call response.
- Prevented a data corruption issue caused by timezone drift by normalizing ingestion to UTC, centralizing serialization rules, and updating **React** reporting surfaces so downstream users saw consistent timestamps across regions.

- Delivered multi-tenant authorization with **OAuth2**, **OIDC**, and **JWT**, then extended **React** route guards and permission-aware UI rendering so privileged actions matched backend **ABAC** rules and immutable audit policies.
- Migrated container delivery from hand-built AMIs to **EKS 1.27** with **Helm 3.12** and **Argo CD 2.8**, while modernizing the frontend delivery pipeline for **React** bundles to support safer rollouts and faster rollback paths.
- Improved throughput and cost efficiency by moving hot reads behind **Redis 6.2** with explicit invalidation triggers, then optimized **React** dashboard hydration patterns so high-traffic operational views loaded faster under peak conditions.
- Standardized deployments with **Terraform 1.6**, adding review gates and promotion rules, and paired those controls with frontend environment validation so **React** releases and infrastructure changes stayed predictable under pressure.
- Built contract-first integration coverage using Pact-style testing across **Python** APIs and **React** consumers, allowing frontend teams and downstream services to release independently without surprise production breakages.

Software Engineer October 2018 to January 2021
Avantia, Inc. - Ohio, United States

- Built reproducible pipeline foundations in **Python 3.8** and delivered internal **React 16–17** operational interfaces that exposed job state, backfill controls, and run metadata without forcing analysts to rely on raw logs.
- Fixed a silent accuracy defect where floating-point rounding changed aggregates by switching to Decimal arithmetic, enforcing scale rules, and updating **React** summary views so finance-facing users saw stable, auditable totals.
- Refactored a legacy ETL monolith into composable jobs with explicit inputs and outputs, then created **React** workflow pages that let operators reprocess failed stages without rerunning entire pipelines.
- Implemented queue-driven execution with **SQS** and worker pools, adding backoff, dead-letter routing, and deduplication, while exposing queue depth and failure counts through lightweight **React** monitoring screens.
- Delivered caching for high-read APIs using **Redis 5.0**, preventing repeated scans on core datasets, and tuned **React** data-fetch behavior so users received fresher information with less UI thrashing during refresh cycles.
- Reduced export failures by validating schemas at service boundaries, emitting typed errors, and reflecting validation states clearly in **React** export tooling so partner data issues were visible within minutes.
- Implemented new export capabilities into **BigQuery** with stable partitioning semantics, and paired them with **React** job-history pages that gave non-technical users searchable visibility into file readiness and load outcomes.
- Strengthened secrets handling through IAM-scoped access and safer local defaults, while removing hard-coded environment assumptions from the **React** build and deployment path to reduce accidental exposure risk.
- Stabilized dashboards by introducing metrics-first instrumentation in **Prometheus 2.x** and alert rules in **Grafana 7.x**, then aligned **React** status indicators with backend health signals for clearer incident awareness.
- Standardized CI behavior using **GitHub Actions** and Dockerized services, and added frontend test coverage around critical **React** flows so release confidence improved across both API and UI changes.
- Partnered with non-technical stakeholders through demos and measurable acceptance checks, translating vague requests into incremental **Python** and **React** releases that avoided scope blowups and improved delivery predictability.

Software Developer March 2016 to September 2018
ArnAmy, Inc. - Florida, United States

- Built ETL routines in **Python 3.6** that normalized event streams into stable identifiers and timestamp semantics, while supporting early **React 16** internal reporting pages that gave teams cleaner access to derived data.
- Fixed a critical mismatch bug where out-of-order events overwrote newer records by introducing monotonic version checks, reconciliation reporting, and UI indicators that highlighted stale versus corrected records for operators.

- Implemented API endpoints in **Django 2.0** with pagination, filtering, and defensive limits, then connected them to internal **React** dataset views so consumers could self-serve information without overloading shared databases.
- Improved query performance in **PostgreSQL 9.6** and **MySQL 5.7** by rewriting hot joins and indexing access paths, while reducing frontend wait time on **React** reporting screens during high-volume searches.
- Shipped a new export and search experience backed by **Elasticsearch 5.6**, including disciplined mappings and analyzers, and surfaced the results through simple **React** interfaces that kept queries predictable for business users.
- Reduced failure time-to-diagnosis by adding structured logging with correlation IDs, then reflected job context and failure summaries inside internal **React** support tooling so operators could triage issues without debugger access.
- Implemented safe retry and timeout policies for upstream integrations, preventing flaky vendor endpoints from cascading into prolonged incidents, and added frontend messaging patterns that made degraded states understandable to users.
- Introduced Dockerized development workflows to reduce environment drift, enabling more consistent API and **React** builds locally and in CI, which improved onboarding and reduced last-minute release surprises.
- Added **Redis**-backed caching for read-heavy paths, defining key strategies and freshness rules carefully, and coordinated frontend invalidation patterns so cached API responses stayed performant without misleading users.
- Turned vague product requests into incremental deliverables with explicit acceptance criteria, shipping practical **Python** backend improvements and lightweight **React** UI enhancements that reduced rework during release weeks.

Junior Software Developer
October 2014 to February 2016

Centric Tech Inc. - New Jersey, United States

- Delivered robust data-cleaning utilities in **Python 2.7** with defensive parsing and repeatable validation outputs, replacing manual spreadsheet firefighting with reviewable, automated processing steps.
- Fixed a recurring ingestion failure caused by inconsistent encodings by normalizing Unicode handling, defining clearer input contracts, and storing sanitized originals so data corrections stayed auditable and repeatable.
- Added safer network handling for early integrations by implementing timeouts, bounded retries, and clearer failure messages, which reduced transient processing issues during peak partner traffic windows.
- Improved release confidence by comparing outputs against known-good baselines, building practical regression checks that caught breaking changes before they reached downstream users or support teams.
- Contributed CI hygiene through scripted checks and more consistent packaging, which reduced “passed locally” surprises and made changes easier for senior engineers to review and roll back safely.
- Helped troubleshoot production incidents using structured logs and targeted query inspection, learning practical bottleneck isolation and documenting fixes so repeat failures became easier to prevent.
- Documented expected inputs and outputs for utilities and jobs so future changes remained understandable, reviewable, and easier for new engineers to extend without breaking shared workflows.
- Collaborated closely with senior engineers to break ambiguous tasks into concrete implementation steps, improving ownership clarity and creating more measurable acceptance criteria on each deliverable.
- Supported early internal web tooling tied to backend utilities, helping connect processed data into simple browser-based views that improved usability for non-technical teammates consuming job outputs.

EDUCATION

Bachelor’s degree in Computer Science
2010 to 2014

Stanford University Department of Computer Science

- Built strong foundations in algorithms, databases, distributed systems, and software engineering, then applied that training through practical team-based projects and production-style system design.